

Security Audit

DelNorte (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	21
Conclusion	22
Our Methodology	23
Disclaimers	25
About Hashlock	26

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The DelNorte team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

DelNorte is a fintech platform that combines blockchain and AI to decentralize the tokenization of real-world assets, enabling secure and transparent transactions between governments, businesses, and individuals. Its ecosystem supports the creation of DAOs (Decentralized Autonomous Organizations) to represent assets like real estate, allowing for fractional ownership and automated governance. DelNorte partners with governments in Latin America to digitize public records, and offers tools like the DelNorte Wallet, Exchange, and Auditor's Network for asset management and verification. The platform aims to modernize public services, enhance investment opportunities, and ensure data integrity through scalable, tamper-proof technology.

Project Name: DelNorte

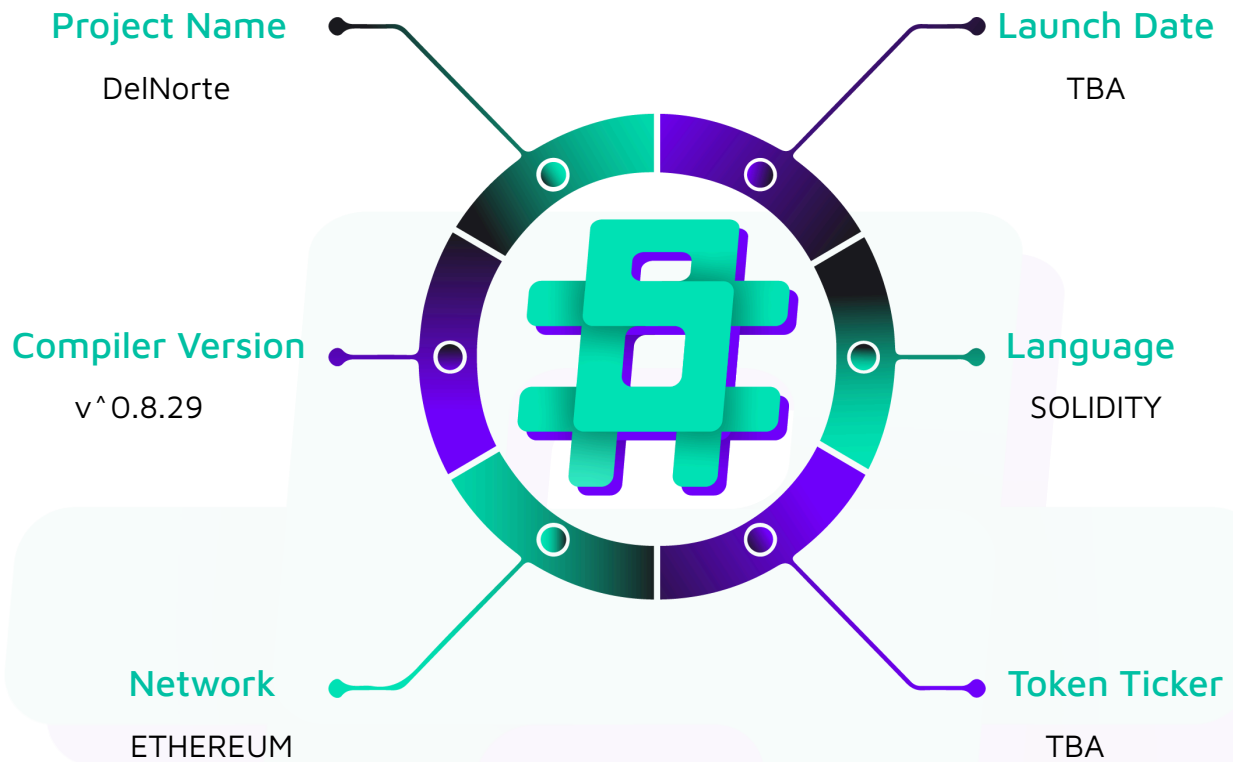
Project Type: Defi

Compiler Version: ^0.8.29

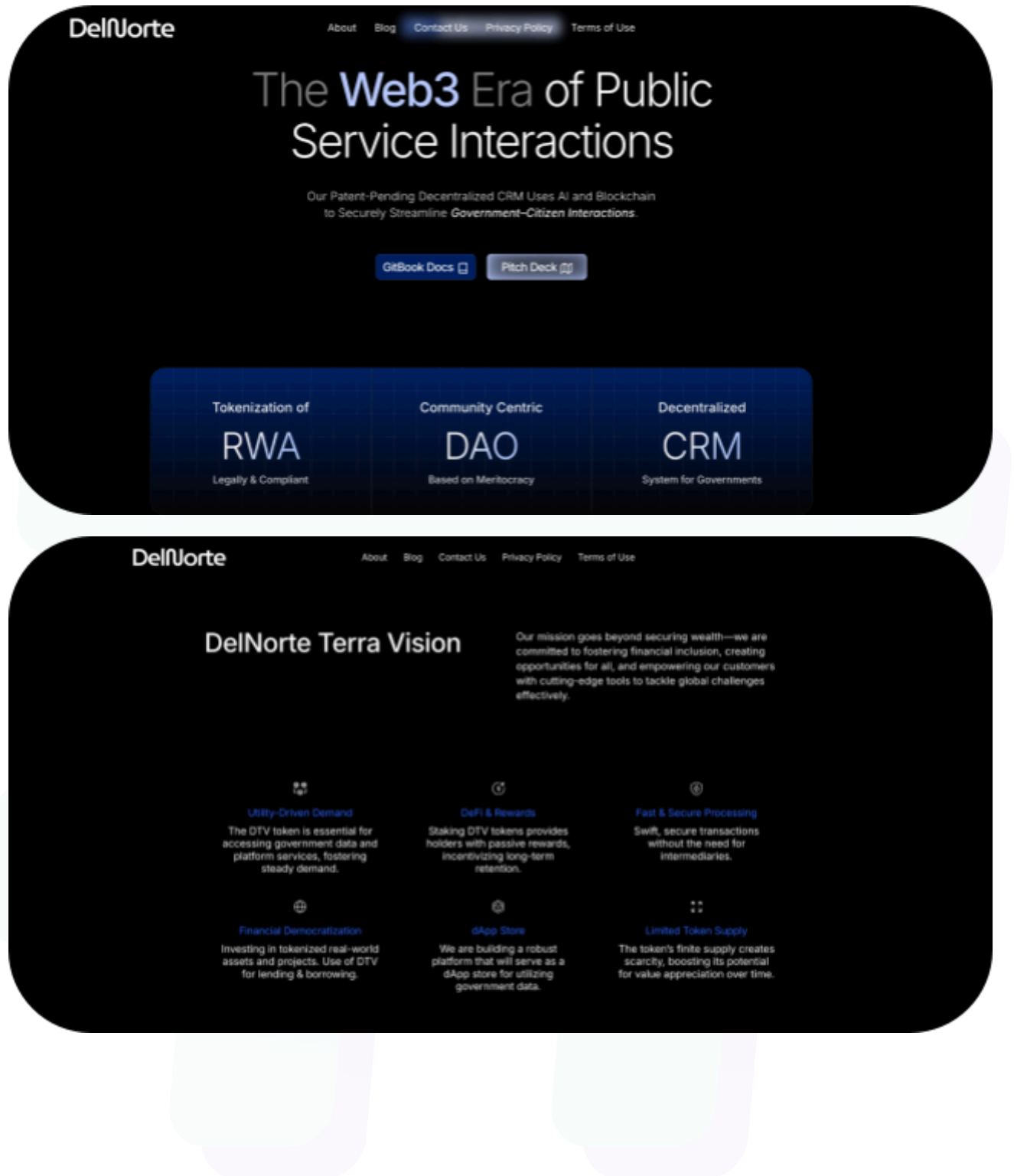
Website: <https://delnorte.io/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the DelNorte project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	DelNorte Smart Contracts
Platform	Ethereum / Solidity
Audit Date	May, 2025
Contract 1	Controller.sol
Contract 1 Address	0x58D3C86dA030B232dE0ACd06FAd21b429A737C50
Contract 2	DelNorteToken.sol
Contract 2 Address	0x91068a9A7681E75EdDD952a2F763C90B98BAc9Ef
Contract 3	DelNorteClubToken.sol
Contract 3 Address	0xe7Ae968823c79CA4022096c0887971358D97ACaa
Contract 4	DelNorteXToken.sol
Contract 4 Address	0x6e50C6b9f6d743F14efD87163D7d36821f57Db12
Contract 5	ReleaseManager.sol
Contract 5 Address	0x09553f6165481ADcD3FEC0725e3408D91bD43e19
Contract 6	TokenSaleManager.sol
Contract 6 Address	0xD7ecEc65586cb27E4dE3317b2f736D015AC885Dc
Contract 7	TokenSwapManager.sol
Contract 7 Address	0xE62Fa65f41E244227655c99BF82538C9E8dc7F89

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 High severity vulnerabilities

2 Medium severity vulnerabilities

1 Low severity vulnerability

1 Gas Optimisation

2 QAs

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
DelNorteToken.sol Functional token of DelNorte El Salvador S.A. and all nations of the Del Norte Network	Contract achieves this functionality.
DelNorteXToken.sol EARLY PURCHASER (more than one year before launch) token of DelNorte El Salvador S.A. and all nations of the Del Norte Network	Contract achieves this functionality.
TokenSaleManager.sol Handles target token sales for USDC, USDT and ETH	Contract achieves this functionality.
TokenSwapManager.sol Handles token swapping and direct swappable token purchases	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the DelNorte project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the DelNorte project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] ReleaseManager#changePeriod - Incorrect event emission of old value

Description

The `changePeriod` function emits an event (`PeriodChanged`) that is intended to log both the old value and the updated value of the `secondsPerPeriod` state variable.

However, the function emits the event after the state variable has already been updated in storage. As a result, the "old value" parameter in the emitted event reflects the newly updated value, rather than the actual old value prior to the update. This behavior misrepresents the intended data in the event and can lead to confusion or incorrect processing by external systems relying on the event logs.

Vulnerability Details

The `changePeriod` function emits the value of the `secondsPerPeriod` state variable for the "old value" parameter but the state variable is already updated.

```
function changePeriod(uint256 newSecondsPerPeriod) external {  
    ...  
    secondsPerPeriod = newSecondsPerPeriod;  
    emit PeriodChanged(secondsPerPeriod, newSecondsPerPeriod);  
}
```

Recommendation

Cache the old value before updating the state variable and emit the cached value.

Status

Resolved

[H-02] ElasticTreasurySpoke#treasuryReclaim, treasuryReclaimETH, PeerTreasury#peerTreasuryTransferETH, peerTreasuryTransferTokens - Incorrect check of reclaimable amount

Description

In the `ElasticTreasurySpoke` contract, the `treasuryReclaim` and `treasuryReclaimETH` functions have checks to ensure that the amount to be reclaimed is less than the `spokeTreasury[tokenAddress].totalReceived` and `spokeTreasuryETH.totalReceived` values.

However, the `spokeTreasury[tokenAddress].totalReceived` and `spokeTreasuryETH.totalReceived` values are only increased when receiving and they are not decreased when reclaiming.

It means that these values could not represent the correct amount of reclaimable funds and checking the amount to be reclaimed with these state values could result in reclaiming more funds than the actual reclaimable amount.

In the `PeerTreasury` contract, the `peerTreasuryTransferETH` and `peerTreasuryTransferTokens` functions have checks to ensure that the amount to be reclaimed is less than the current balances of funds stored in the contract.

However, the balance of the contract also contains the funds received through the `ElasticTreasury`'s receive functions.

It means that the actual balance of the contract could be greater than the actual reclaimable amount of the `PeerTreasury` and checking the amount to be reclaimed with the balances could result in reclaiming more funds than the actual reclaimable amount.

Recommendation

Calculate the reclaimable amount by subtracting `spokeTreasury[tokenAddress].totalReclaimed/spokeTreasuryETH.totalReclaimed` from `spokeTreasury[tokenAddress].totalReceived/spokeTreasuryETH.totalReceived` and then compare with the amount to be reclaimed in the `ElasticTreasurySpoke` contract.

In the `PeerTreasury` contract, compare the amounts to be reclaimed with the return values of the `peerETHAvailable` and `peerTokenAvailable` functions.

Status

Resolved



Medium

[M-01]TokenSaleManager#buyWithETH,TokenSwapManager#buySwappableTokenWithETH - Lack of freshness check for external price feed data

Description

The smart contracts retrieve the eth price data from an external price feed contract but do not implement a freshness check to ensure the data is up-to-date. This omission introduces a significant risk, as the contract may rely on outdated or stale price data, which can lead to incorrect calculations, financial losses, or exploitation by malicious actors.

Recommendation

Introduce a max allowed timeline and Implement a freshness check mechanism to validate the timeliness of the price data before using it in the contract's logic.

Status

Resolved

[M-02]Recoverable#reverseAccidentERC20,ElasticTreasurySpoke#treasuryReclaim,TokenSwapManager#buySwappableTokenWithStableCoin,buySwappableTokenWithUSDCPermit, buySwappableTokenWithETH - Missing checks for return value

Description

The functions do not verify the return value of the `transfer` functions when transferring the ERC-20 tokens.

According to the ERC-20 standard, the `transfer` function is expected to return a boolean value indicating whether the operation was successful. However, some ERC-20 tokens do not strictly adhere to the standard and may fail silently without reverting the transaction. This can lead to scenarios where the transfer fails, but the smart contract continues execution as if the transfer succeeded, potentially causing loss of funds or unexpected behavior.

Recommendation

It is recommended to check the return value of the `transfer` function to ensure the transfer was successful.

Status

Resolved

Low

[L-01] DelNorteClubToken#setGating, TokenSaleManager#setMinDollarValue, setDefaultUSCliff, setDefaultNonUSCliff - Important functions lack event emissions

Description

Critical functions should emit events to provide transparency and enable off-chain monitoring of important state changes. Without events, it becomes difficult to track and verify contract operations, complicating both user interfaces and protocol monitoring. Instances are shown below.

```
function setGating() // DelNorteClubToken.sol  
function setDefaultUSCliff() // TokenSaleManager.sol  
function setMinDollarValue() // TokenSaleManager.sol  
function setDefaultNonUSCliff() // TokenSaleManager.sol
```

Recommendation

Review your contracts to implement event emissions for critical state variable updates.

Status

Resolved

Gas

[G-01] ElasticTreasurySpoke, PeerTreasury - State variable that could be made immutable

Description

The controller, hubTokenAddress, and peerTreasuryController variables are only set in the constructor and never changed anymore.

Such variables could be made immutable to save gas when reading them.

Recommendation

Make the variables immutable.

Status

Resolved

QA

[Q-01] Contracts - Use of floating pragma

Description

Contracts use a pragma statement that does not specify a fixed compiler version but instead allows the contract to be compiled with any compatible compiler version. This can lead to various compatibility and stability issues.

Recommendation

Consider locking the pragma version whenever possible and avoid using a floating pragma in the final deployment. Consider [known bugs](#) for the compiler version that is chosen.

Status

Resolved

[Q-02] TokenSaleManager - Redundant assignment of a default value

Description

The `startSaleBlockNum` variable is explicitly set to 0 when it's declared.

However, since 0 is the default value for this variable, this assignment is redundant.

Recommendation

Remove the redundant assignment of a default value.

Status

Acknowledged

Centralisation

The DelNorte project prioritizes decentralization alongside security and utility

One of the defining aspects of the Del Norte Protocol's architecture is its inherent flexibility to progressively transition toward full decentralization. Although the current implementation includes an executive configuration, the protocol is not restricted to an "owner-type" governance structure. It has been designed to support the complete removal of centralized control, including any executive roles, at any point. This enables governance and operational authority to be seamlessly transferred to the community or a decentralized autonomous organization (DAO) framework without technical constraints.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the DelNorte project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.